



SELF LEARNING

Presented by Pratyush Kr. Singh

order_details

**DATASET
OVERVIEW**

orders

**pizzas /
pizza_types**



PROJECT SNAPSHOT

This workbook contains 12 SQL queries written against the Pizzahut sales database, covering everything from basic aggregation to window functions and subqueries. The goal was to practice real-world business analysis using SQL order volume, revenue, product performance, and time-based trends. All queries were written and tested in MySQL Workbench



PROJECT FOCUS



Sales & order analysis — total orders, total revenue, and identifying the highest-priced and most-ordered pizzas.



Trend & category analysis — order distribution by hour, revenue contribution by category, and cumulative revenue over time using window functions.



MAIN OBJECTIVES

Practice core SQL —
COUNT, SUM, GROUP BY,
and basic filtering.



Apply intermediate
techniques multi-table
JOINS, subqueries, and
ranking with ORDER BY /
LIMIT.

Master advanced
concepts — correlated
subqueries and window
functions (SUM() OVER).



PROJECT FLOW

- Explored the schema (orders, order_details, pizzas, pizza_types) and their relationships.
- Wrote 12 progressively harder queries — from basic counts to window functions.
- Validated each result against the dataset in MySQL Workbench.



WORK STRATEGY



1.

Basic: Found total orders (21,350), total revenue (₹8,17,860.05), and the highest-priced pizza (The Greek Pizza, ₹35.95).

2.

Intermediate: Identified the most ordered size (Large, 18,526 orders), top 5 pizzas by quantity, and category-wise order counts.

3.

Advanced: Calculated average pizzas ordered per day (138), % revenue contribution by category, and cumulative revenue over time.

```

1  -- Retrive the total numbers of order placed
2
3  •  select count(order_id) as total_orders from orders;

```

BASIC

OUTPUT

COUNT() — Total orders: 21,350

Result Grid	
	total_orders
▶	21350

Result Grid

Filter R...

	category	quantity
▶	Classic	14888
	Supreme	11987
	Veggie	11649
	Chicken	11050

OUTPUT

JOIN + GROUP BY — Classic led with 14,888 units

```

1  -- Join the necessary tables to find the total quantity of each pizza category ordered
2  •  SELECT
3      pizza_types.category, SUM(order_details.quantity) AS quantity
4  FROM
5      pizza_types
6      JOIN
7      pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
8      JOIN
9      order_details ON order_details.pizza_id = pizzas.pizza_id
10 GROUP BY pizza_types.category
11 ORDER BY quantity DESC;

```

INTERMEDIATE

```

1  -- analyze the cumulative revenue generted over time
2
3
4  •  select order_date, sum(revenue) over(order by order_date) as cum_revenue
5  from
6
7  (select orders.order_date, sum(order_details.quantity*pizzas.price) as revenue
8  from order_details join pizzas
9  on order_details.pizza_id = pizzas.pizza_id
10 join orders
11
12 on orders.order_id =order_details.order_id
13 group by orders.order_date)as sales;

```

ADVANCED

OUTPUT

	order_date	cum_revenue
▶	2015-01-01	2713.85000000000004
	2015-01-02	5445.75
	2015-01-03	8108.15
	2015-01-04	9863.6
	2015-01-05	11929.55
	2015-01-06	14358.5

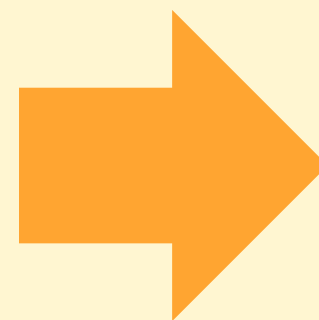
Window function — Running revenue total by date



TASK DIVISION



Started with simple SELECT, COUNT, and SUM queries on single tables to answer basic business questions.



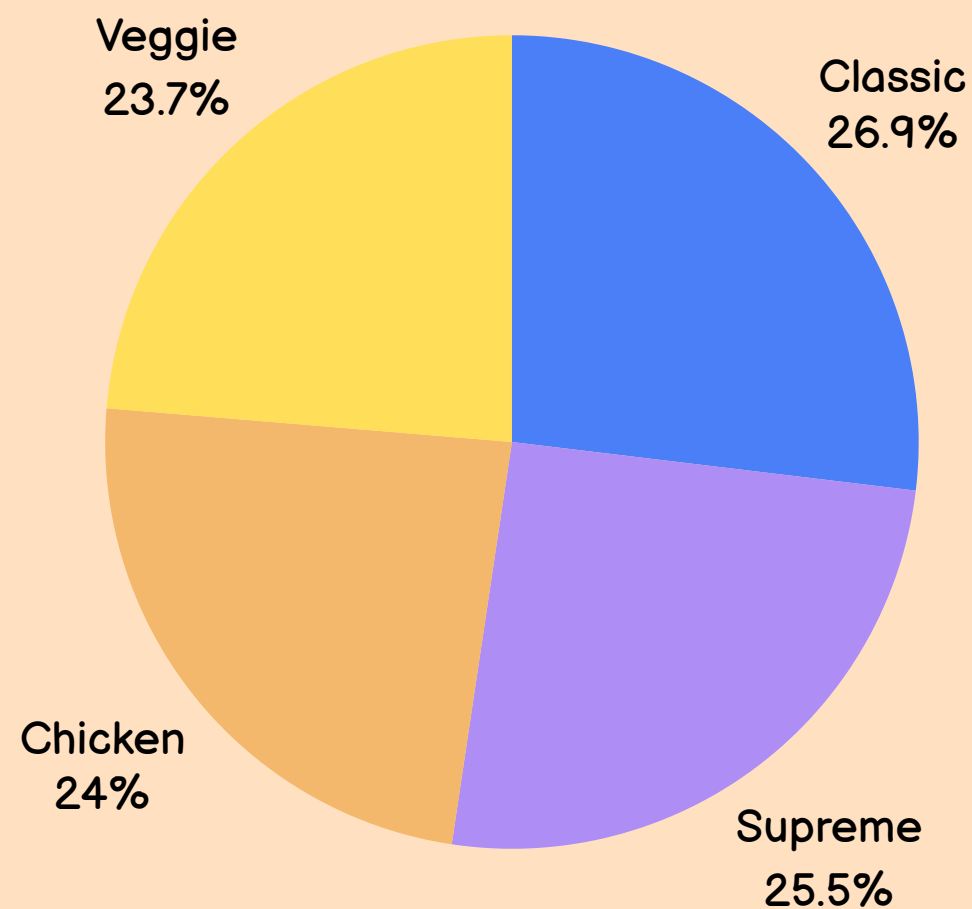
Progressed to multi-table JOINS, subqueries, and window functions to answer deeper, trend-based questions.

OUTCOME REVIEW

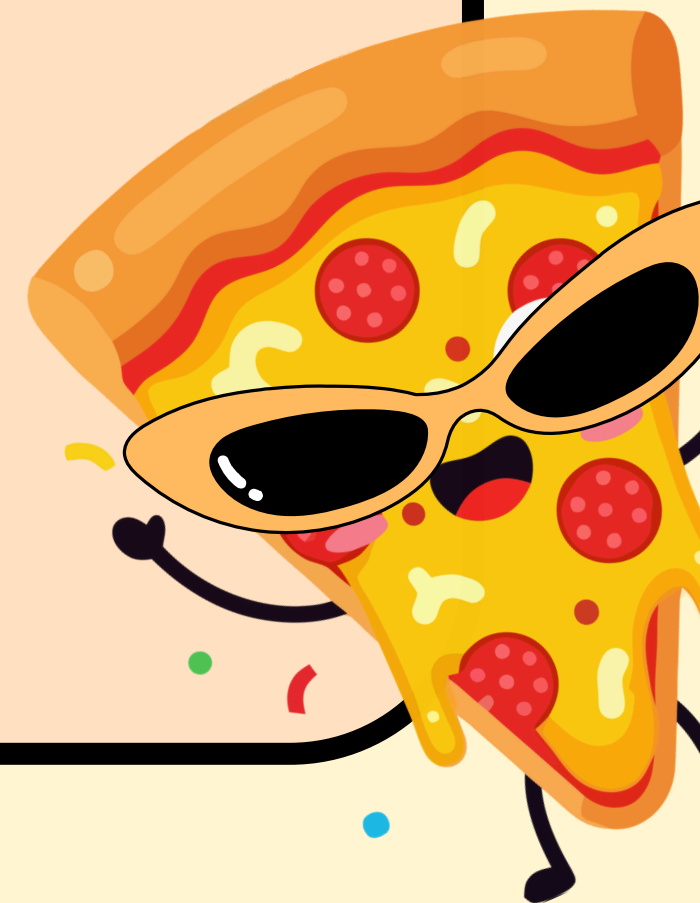
Through this workbook I strengthened my ability to translate business questions into SQL — calculating revenue, ranking top-performing products, analyzing time-based order patterns, and using window functions for running totals. This built confidence in writing production-style analytical queries, not just syntax practice.



FINAL WRAP



Classic pizzas led revenue contribution at ~27%, closely followed by Supreme, Chicken, and Veggie — a fairly balanced category split across the menu.



THANK YOU

Presented by Pratyush Kr. Singh

